



Spring JMX

1

Spring JMX

2

- ✓ Introduction
- ✓ Architecture JMX
- ✓ Mbean
- ✓ Annotations Spring JMX
- ✓ Gestion des notifications

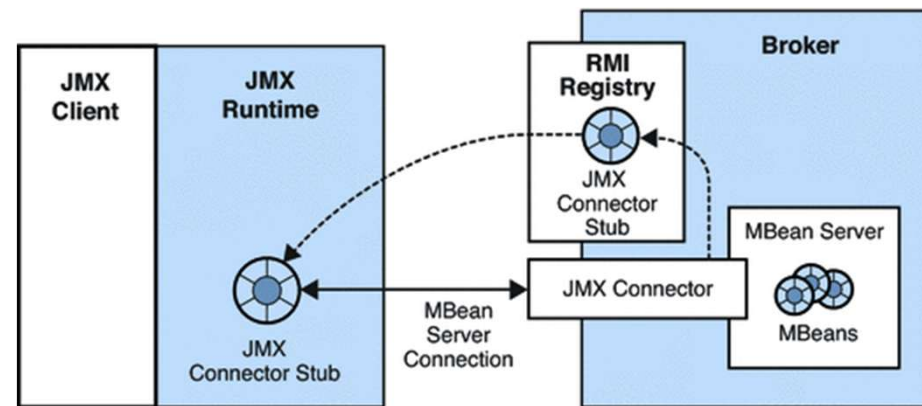
Présentation

- **Java Management eXtensions** : anciennement Java Management API.
- Via JMX on peut mettre en place des objets qui pourront surveiller et gérer une application à distance.
- La plupart des principaux serveurs d'applications Java EE utilisent JMX pour la surveillance et la gestion de leurs composants.
 - Tomcat, Jboss, ...

JMX Architecture

➤ L'architecture JMX se compose souvent de la manière suivante :

- Un bean JMX, souvent appelé MBean, composé de méthodes accessibles ou non à distance
- Une interface pour le bean JMX
- Une RMI Registry pour placer à disposition le bean



JMX - MBean

5

- Le MBean est un objet Java normal qui n'a rien de particulier hormis le fait que c'est un bean :
 - Doit être une classe publique
 - Doit avoir au moins un constructeur public sans paramètre
 - Doit être sérialisable ⇔ implémenter l'interface `java.io.Serializable`

JMX - MBean

- Il n'y a pas de contrainte sur son nom de classe ou celui du package
 - Evitez cependant de les appeler XxxXMBean
- Il n'y a pas de contrainte sur ses attributs
- Il n'y a pas de contrainte sur ses méthodes
- Il peut faire ce que vous voulez
 - Gardez cependant en tête que JMX a pour objectif de **monitorer** et **piloter** une application à travers ses Mbeans
 - De nos jours, un bean JMX **ne doit pas** être utilisé comme un service métier distant

Interface du MBean

- Dans la norme standard, le MBean **doit** être représenté par une interface Java
- Cette interface va exposer
 - les informations : méthodes get/set
 - les fonctionnalités : autres méthodes
- Dans la norme, elle doit **impérativement** porter le nom du bean suivi par **MBean**

Exemple

8

La classe

```
1 package fr.jmx;
2
4+ * Classe representant une calculatrice. <br/>[]
8 public class Calculatrice implements CalculatriceMBean {
9     private double dernierResultat;
10
12+ * Constructeur.[]
14- public Calculatrice() {
15     super();
16 }
17
18- @Override
19 public double add(double c1, double c2) {
20     double resultat = c1 + c2;
21     this.dernierResultat = resultat;
22     return resultat;
23 }
24
26+ public double sub(double c1, double c2) {[]
31
33+ public double mult(double c1, double c2) {[]
38
40+ public double div(double c1, double c2) {[]
48
50+ public double getDernierResultat() {[]
53 }
```

L'interface : *NomDeClasseMBean*

```
1 package fr.jmx;
2
4+ * Interface representant une calculatrice. <br/>[]
8 public interface CalculatriceMBean {
9
11+ * Additionne deux valeurs.[]
19 public abstract double add(double c1, double c2);
20
22+ * Soustrait deux valeurs.[]
30 public abstract double sub(double c1, double c2);
31
33+ * Multiplie deux valeurs.[]
41 public abstract double mult(double c1, double c2);
42
44+ * Divise deux valeurs.[]
52 public abstract double div(double c1, double c2);
53
55+ * Recupere le dernier resultat[]
59 public abstract double getDernierResultat();
60
61 }
```


Mise à disposition du MBean

- Pour mettre à disposition le MBean, on utilise une RMI Registry
- La RMI Registry est un **annuaire** dans lequel on peut placer des objets
 - C'est un processus qui se lance
 - Elle aura une adresse, un URL avec un numéro de port
- Chacun des objets a une adresse unique
 - Le client doit connaître l'adresse de la RMI Registry
 - Le client doit connaître l'adresse du Mbean dans la RMI Registry

Exemple : côté serveur

Lance une RMI Registry et met à disposition un bean dedans

```
1 package fr;
2
3 import java.lang.management.ManagementFactory;
4
5 import javax.management.MBeanServer;
6 import javax.management.ObjectName;
7
8 import fr.jmx.Calculatrice;
9 import fr.jmx.CalculatriceMBean;
10
11 * Classe qui fabrique le bean JMX, le reference et attend. <br/>
12 public class Run {
13
14     * Test qui fabrique un bean JMX et le laisse a disposition.
15     public static void main(String[] args) {
16         // Recupere l'annuaire de bean JMX
17         MBeanServer mbs = ManagementFactory.getPlatformMBeanServer();
18         try {
19             // Nom de l'objet
20             ObjectName name = new ObjectName("fr.jmx:type=Calculatrice");
21             // Construction de l'objet
22             CalculatriceMBean mbean = new Calculatrice();
23             // Inscription de l'objet dans l'annuaire
24             mbs.registerMBean(mbean, name);
25             // On attend
26             System.out.println("Maintenant, ouvrez un shell et lancez jconsole");
27             System.out.println("N'oubliez pas de tuer le processus a la fin (carre rouge)");
28             Thread.sleep(Long.MAX_VALUE);
29         } catch (Exception e) {
30             e.printStackTrace();
31         }
32     }
33 }
```

JMX : ObjectName

- Chaque MBean possède un ObjectName qui permet de l'identifier.
- La syntaxe d'un ObjectName est de la forme [nomDomain]:propriete=valeur[,propriete="valeur"]*
 - un nom de domaine
 - ❑ Peut être vide
 - ❑ En général, on prend le nom du package, mais ce n'est pas une obligation, on peut aussi prendre le nom de l'application
 - ❑ Vous ne pouvez pas utiliser les '/' ou ':'
 - ❑ '*' et '?' sont réservés pour la recherche
 - un ensemble de propriétés sous la forme de paires clé = valeur séparées par des virgules.
 - ❑ Au moins une propriété doit être définie
 - ❑ Vous pouvez faire usage de " " pour entourer les valeurs (quand il y a une ',' par exemple)
 - ❑ L'ordre des propriétés n'est pas significatif.
- Attention : les **espaces** contenus dans l'ObjectName sont significatifs et ils sont aussi **sensibles à la casse**.

JMX : Côté client

- Pour accéder à votre Mbean vous aurez besoin :
 - De l'adresse de la RMI Registry
 - De l'object Name de votre MBean
- Vous pouvez accéder à votre bean via :
 - Du code
 - Un client déjà fournit par Oracle : la JConsole

JMX : Côté client

- La JConsole se trouve dans le dossier bin de votre JVM
 - Il existe sous toutes les plateformes
- Pour le lancer :
 - Sous Windows, Contrôle de Mission Java 
 - ❑ Puis accédez à la console JMX
 - Sous Shell, [JAVA_HOME]\bin\jconsole
- Remarque : si vous avez des difficultés à trouver votre processus Java, lancez la jconsole en tant qu'administrateur

JMX : Côté client



14

Processus
Java

ObjectName

Opérations

Oracle Java Mission Control

File Edit Window Help

JVM B... Event ...

[1.8.0_144] Eclipse (584)
[1.8.0_144] The JVM Running
[1.8.0_144] fr.Run (6880)
MBean Server
Flight Recorder

MBean Browser

Filter:

MBean Tree

- JMImplementation
- com.sun.management
- fr.jmx
 - Calculatrice
- java.lang
- java.nio
- java.util.logging

MBean Features

Attributes Operations Notifications Metadata

Operations

- add : double
- div : double
- mult : double
- sub : double

Name	Value	Description
p1	5.0	
p2	10.0	

Execute add(5.0, 10.0)

Overview MBean Browser Triggers System Memory Threads Diagnostic Commands

Mbeans
du
processus

JMX

15

➤ Plus d'information sur le JMX :

- <https://www.jmdoudoux.fr/java/dej/chap-jmx.htm>
- <https://docs.oracle.com/javase/tutorial/jmx/>

Spring et JMX

- Spring va nous permettre de :
 - Déclarer des MBean
 - Déclarer une RMI Registry
 - Récupérer des MBean

- Sans avoir à écrire trop de code
 - Mais en faisant un peu de configuration

Spring et JMX : MBean

- En Spring, un MBean suit les mêmes contraintes que dans la norme Java JMX
- Cependant, vous n'êtes pas dans l'obligation de le représenter par une interface
 - C'est tout de même très fortement recommandé, mais ce n'est pas une obligation

Spring et JMX : Interface du MBean

- L'interface qui va représenter votre MBean n'a pas obligation de se nommer XxxMBean
- Vous pouvez conserver le nom XxxMBean si vous le souhaitez mais ce n'est plus une obligation en Spring

Spring et JMX : RMI Registry

- C'est le Spring qui va se charger :
 - De la création de la RMI Registry
 - De sa mise à disposition, son démarrage
 - De l'ajout de votre MBean dans la registry
- Rien ne se fait dans le code, tout se fera dans le fichier de configuration

Spring et JMX : RMI Registry

20

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <beans xmlns="http://www.springframework.org/schema/beans"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4     xmlns:context="http://www.springframework.org/schema/context"
5     xsi:schemaLocation="http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans/spring-beans.xsd
6         http://www.springframework.org/schema/context http://www.springframework.org/schema/context/spring-context.xsd">
7
8     <!-- On declare un registre RMI -->
9     <bean id="rmiRegistry" class="org.springframework.remoting.rmi.RmiRegistryFactoryBean">
10         <property name="port" value="10099" />
11     </bean>
12
13     <!-- On indique l'url ou sera disponible notre bean service:jmx:rmi://localhost/jndi/rmi://localhost:10099/myconnector -->
14     <bean class="org.springframework.jmx.support.ConnectorServerFactoryBean" depends-on="rmiRegistry">
15         <property name="serviceUrl"
16             value="service:jmx:rmi://localhost/jndi/rmi://localhost:10099/myconnector" />
17     </bean>
18
19 </beans>
```

- depends-on : indique que le bean (ici *ConnectorServerFactoryBean*) sera fabriqué **APRES** celui représenté par l'id donné (ici *RmiRegistryFactoryBean*).

Spring et JMX : Déclaration MBean

- Après avoir codé le MBean (et idéalement son interface), Spring va se charger de les monter dans la RMI registry

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <beans xmlns="http://www.springframework.org/schema/beans"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4     xmlns:context="http://www.springframework.org/schema/context"
5     xsi:schemaLocation="http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans/spring-beans.xsd
6         http://www.springframework.org/schema/context http://www.springframework.org/schema/context/spring-context.xsd">
7
8     <!-- Declaration du bean JMX -->
9     <bean id="jmxCalculatrice" class="fr.jmx.Calculatrice">
10         <description>Bean calculatrice</description>
11     </bean>
12
13     <!-- On export notre bean -->
14     <bean class="org.springframework.jmx.export.MBeanExporter">
15         <property name="beans">
16             <map>
17                 <entry key="monApplication:type=mesBeans,name=MaCalculatrice" value-ref="jmxCalculatrice" />
18             </map>
19         </property>
20         <property name="assembler">
21             <bean class="org.springframework.jmx.export.assembler.SimpleReflectiveMBeanInfoAssembler" />
22         </property>
23     </bean>
24
25 </beans>
```

Spring et JMX : Déclaration MBean

- La mise à disposition du MBean se fait par l'objet Spring `org.springframework.jmx.export.MBeanExporter`
- Il prend une Map :
 - **Clef** : ObjectName de votre bean JMX (attribut key ou key-ref)
 - **Valeur** : Votre bean JMX (attribut value ou value-ref)
- Il prend aussi un objet *assembler* qui devra appartenir à la famille des `org.springframework.jmx.export.assembler.MBeanInfoAssembler`

Spring et JMX : Déclaration MBean

- Pour votre *assembler*, vous avez le choix :
 - **SimpleReflectiveMBeanInfoAssembler** : expose par introspection toutes les propriétés et les méthodes publiques
 - **InterfaceBaseMBeanInfoAssembler** : expose les propriétés et les méthodes du MBean définies dans une interface
 - **MetadataMBeanInfoExporter** : utilise des annotations pour déterminer les propriétés et les opérations du MBean à exposer (c'est l'implémentation par défaut)
 - **MethodExclusionMBeanInfoAssembler** : expose toutes les propriétés et les méthodes du MBean sauf celles précisées
 - **MethodNameBasedMBeanInfoAssembler** : vous permet d'indiquer les méthodes à exposer. Toutes les méthodes get/set sont exposées automatiquement.

Spring et JMX : Déclaration MBean

➤ Autre exemple avec un autre assembleur

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <beans xmlns="http://www.springframework.org/schema/beans"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:context="http://www.springframework.org/schema/context"
4     xsi:schemaLocation="http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans/spring-beans.xsd
5                         http://www.springframework.org/schema/context http://www.springframework.org/schema/context/spring-context.xsd">
6
7     <!-- Declaration du bean JMX -->
8     <bean id="jmxCalculatrice" class="fr.jmx.Calculatrice">
9         <description>Bean calculatrice</description>
10    </bean>
11
12    <!-- On export notre bean -->
13    <bean class="org.springframework.jmx.export.MBeanExporter">
14        <property name="beans">
15            <map>
16                <entry key="monApplication:type=mesBeans,name=MaCalculatrice" value-ref="jmxCalculatrice" />
17            </map>
18        </property>
19
20        <property name="assembler">
21            <bean class="org.springframework.jmx.export.assembler.MethodNameBasedMBeanInfoAssembler">
22                <property name="managedMethods">
23                    <list>
24                        <value>add</value>
25                        <value>sub</value>
26                    </list>
27                </property>
28            </bean>
29        </property>
30    </bean>
31 </beans>
```


Spring et JMX : Côté client

- Vous pouvez toujours utiliser la JConsole
- Vous pouvez aussi passer par du code
 - Spring se chargera d'aller chercher votre Mbean
 - Vous n'avez pas de code à écrire
 - Tout sera dans le fichier de configuration

Spring et JMX : Côté client

- Vous devrez déclarer dans votre fichier de configuration Spring côté client, deux éléments :
 - org.springframework.jmx.support.**MBeanServerConnectionFactoryBean** : qui portera la *configuration* de la RMI Registry
 - org.springframework.jmx.access.**MBeanProxyFactoryBean** : qui portera votre MBean
 - ❑ Une déclaration par MBean

Spring et JMX : Côté client

➤ Exemple :

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <beans xmlns="http://www.springframework.org/schema/beans"
3       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:context="http://www.springframework.org/schema/context"
4       xsi:schemaLocation="http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans/spring-beans.xsd
5                           http://www.springframework.org/schema/context http://www.springframework.org/schema/context/spring-context.xsd">
6
7   <bean id="serverConnection" class="org.springframework.jmx.support.MBeanServerConnectionFactoryBean">
8     <!-- On indique l'URL du serveur RMI que l'on a cree pour notre bean -->
9     <property name="serviceUrl"
10       value="service:jmx:rmi://localhost/jndi/rmi://localhost:10099/myconnector" />
11   </bean>
12
13   <bean id="calculatriceJMXProxy" class="org.springframework.jmx.access.MBeanProxyFactoryBean">
14     <property name="server" ref="serverConnection" />
15     <!-- On replace l'object name que l'on a donne a notre bean JMX -->
16     <property name="objectName" value="monApplication:type=mesBeans,name=MaCalculatrice" />
17     <property name="proxyInterface" value="fr.jmx.ICalculatrice" />
18   </bean>
19
20 </beans>
```

➤ Notez ici l'intérêt d'avoir une interface pour représenter notre MBean côté client

Travaux pratiques

- Exo 22 - Réalisé le premier TP JMX
 - ➡ Réalisation d'une calculatrice en JMX



Spring et JMX : Annotations

- Vous pouvez vous simplifier la vie et faire usage des annotations lors de la réalisation de vos MBean
- Les annotations se chargeront de déclarer :
 - votre bean dans le contexte Spring
 - les méthodes gérées par JMX
 - Nom, description, description des paramètres, ...
 - L'ObjectName de votre MBean
- L'utilisation des annotations simplifiera votre fichier de configuration Spring

Spring et JMX : Annotations

➤ Annotations importantes disponibles :

- **@ManagedResource** : s'utilise sur la classe du bean pour déclarer qu'une classe est un MBean
- **@ManagedAttribute** : s'utilise sur une propriété du bean pour la définir comme exposable (get+set)
- **@ManagedOperation** : s'utilise sur une méthode du bean pour la définir comme exposable
- **@ManagedOperationParameters** : s'utilise sur une méthode du bean pour la définir le tableau des **@ManagedOperationParameter**
 - ❑ **@ManagedOperationParameter** : s'utilise dans un **@ManagedOperationParameters**, permet de décrire un paramètre de méthode

Spring et JMX : Annotations

- **@ManagedMetric** : s'utilise sur une méthode de type get. Permet d'exposer une propriété sous la forme d'un *metric* JMX.
- **@ManagedNotifications** : s'utilise sur la classe pour indiquer le tableau de @ManagedNotification
 - ❑ **@ManagedNotification** : s'utilise dans un @ManagedNotifications, permet de décrire la notification émise par le MBean
 - ❑ Ne vous épargne en rien l'écriture du code associé à l'envoi des notifications

Spring et JMX : Annotations

```
1 package fr.jmx;
2
3 import org.apache.log4j.LogManager;
4 import org.apache.log4j.Logger;
5 import org.springframework.jmx.export.annotation.ManagedAttribute;
6 import org.springframework.jmx.export.annotation.ManagedOperation;
7 import org.springframework.jmx.export.annotation.ManagedOperationParameter;
8 import org.springframework.jmx.export.annotation.ManagedOperationParameters;
9 import org.springframework.jmx.export.annotation.ManagedResource;
10 import org.springframework.stereotype.Component;
11
12 * Classe representant une calculatrice. <br/>
13 @Component
14 @ManagedResource(objectName = "monApplication:type=messBeans,name=MaCalculatrice", description = "Ma calculatrice JMX")
15 public class Calculatrice implements ICalculatrice {
16     private static final Logger LOG = LogManager.getLogger(Calculatrice.class);
17
18     private double dernierResultat;
19
20     * Constructeur.
21     public Calculatrice() {}
22
23     @Override
24     @ManagedOperation(description = "Additionne deux chiffres")
25     @ManagedOperationParameters({ @ManagedOperationParameter(name = "c1", description = "Le premier chiffre"),
26         @ManagedOperationParameter(name = "c2", description = "Le second chiffre") })
27     public double add(double c1, double c2) {
28         Calculatrice.LOG.debug(c1 + "+" + c2);
29         double resultat = c1 + c2;
30         this.dernierResultat = resultat;
31         return resultat;
32     }
33 }
```


Spring et JMX : Annotations

- Votre fichier de configuration (côté serveur) peut se simplifier :

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <beans xmlns="http://www.springframework.org/schema/beans"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:context="http://www.springframework.org/schema/context"
4     xsi:schemaLocation="http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans/spring-beans.xsd
5                         http://www.springframework.org/schema/context http://www.springframework.org/schema/context/spring-context.xsd">
6
7     <context:component-scan base-package="fr.jmx"/>
8
9     <!-- A la place du parametrage detaille on peut ecrire -->
10    <context:mbean-export/>
11
12 </beans>
```

- Attention : vous devrez revenir à une déclaration plus explicite du **MBeanExporter** si vous voulez gérer les évènements.

Travaux pratiques

- Exo 22 - Reprenez le TP précédent et faites usage des annotations



Spring et JMX : Evènements

- Un bean JMX peut
 - Envoyer des évènements (ou notifications)
 - Etre à l'écoute d'évènements (ou notifications)
- Vous allez retrouver ici le pattern du Listener très fortement utiliser dans la gestion des interfaces graphiques (Swing, Android, ...)

Spring et JMX : Evènements

- L'objet (qui n'est pas obligatoirement un MBean) qui veut être à l'écoute devra implémenter l'interface
 - **javax.management.NotificationListener**
 - `public void handleNotification(Notification, Object)`
 - A vous d'y décrire ce que doit faire cet objet quand il capte une notification
- Si il le souhaite, il peut aussi déclarer le fait qu'il souhaite filtrer les notifications qui l'intéresse, dans ce cas il implémentera aussi l'interface
 - **javax.management.NotificationFilter**
 - `public boolean isNotificationEnabled(Notification)`
 - A vous d'y décrire le type de notification qui intéresse l'objet

Spring et JMX : Evènements

- Le MBean qui souhaite envoyer des notifications devra :
 - Ajouter un attribut de type
`org.springframework.jmx.export.notification.NotificationPublisher`
 - Implémenter l'interface
`org.springframework.jmx.export.notification.NotificationPublisherAware`
 - ❑ `public void setNotificationPublisher(NotificationPublisher)`
 - Qui initialisera l'attribut `NotificationPublisher`
 - Définir quand et où envoyer ses notifications
 - ❑ A travers la méthode `sendNotification` de son attribut `NotificationPublisher`

Spring et JMX : Evènements

➤ Côté configuration Spring, via le MBeanExporter et sa propriété `notificationListenerMappings` il vous faudra indiquer :

➡ Les MBean ciblés par vos/votre Listener

- ☐ Vous pouvez faire usage de « * » pour indiquer tous les beans
- ☐ Sinon, vous pouvez indiquer l'ObjectName du MBean

```
<property name="notificationListenerMappings">
  <map>
    <entry key="*">
      <bean class="fr.jmx.log.LogNotificationListener"/>
    </entry>
  </map>
</property>
```

Travaux pratiques

- Exo 22 - Reprenez le TP précédent
 - Ajoutez un listener qui loguera simplement les informations qui lui arrivent
 - Remarque : Si vous avez fait usage du `context:mbean-export`, il vous faudra revenir à une écriture plus explicite.



Spring Boot Actuator

- Il existe un module dans Spring Boot qui permet de monitorer votre application via JMX
 - `spring-boot-starter-actuator`
- Son activation vous donnera des informations sur l'état de global du projet
 - Vous pouvez aussi y inclure vos propre éléments de mesure
- <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/#actuator>